pyMobaLedLib Web-Oberfläche – Installation und Betrieb

Windows · macOS · Linux-Desktop · Raspberry Pi

Datei-Version: 2.24 (2026-10-15)

Inhalt

Schnellstart für Eilige

1. Welches System, welches Skript?

2. Gemeinsame Grundlagen

2.1 Dateien und Ordnerstruktur

2.2 Die ARDUINO IDE: Version 1.8.19, nicht 2.x

2.3 Bibliotheken, Boards und die Tabelle „Libraries“

2.4 Starten, Beenden, Speichern

2.5 Das Programmsymbol

2.6 Wo liegen meine Daten? (Datenordner)

2.7 Protokoll (Log)

2.8 Kleine Bildschirme: kompaktes Layout

3. Windows

4. macOS

5. Linux-Desktop (Debian, Linux Mint, Ubuntu, Fedora, Arch …)

6. Raspberry Pi (Dauerdienst)

6.1 Dateien auf den Pi bringen

6.2 Installation (ein Befehl)

6.3 Das Desktop-Symbol

6.4 ARDUINO IDE und Bibliotheken auf dem Pi

6.5 Alltag auf dem Pi

6.6 Raspberry Pi herunterfahren

6.7 Den Raspberry Pi im Netz finden

7. Aktualisieren und Deinstallieren

7.1 Update über das Programm (Hilfe → Update webmobaledlib …)

8. Zugriff aus dem Netz und Passwort

9. Fehlersuche

9.1 Alle Systeme

9.2 Windows

9.3 macOS

9.4 Linux-Desktop

9.5 Raspberry Pi

10. Was nicht oder noch nicht geht

Der Programmgenerator läuft als Web-Oberfläche: Das Programm startet auf dem Rechner, an dem der Arduino per USB hängt, und du bedienst es im Browser (Edge, Chrome, Firefox, Safari). Es benutzt dieselben Programmdateien wie die Tk-Version. Diese Anleitung gilt für alle vier Systeme. **Eilige lesen nur den Schnellstart.** Wer es genauer wissen will, liest Kapitel 1 und 2 und danach das Kapitel für sein System. Betatester finden eine Checkliste in `install/BETATEST_Checkliste.md`.

Schnellstart für Eilige

Vorbereitung (für alle Systeme)

1. Die Datei **`webmobaledlib.zip`** aus dem **neuesten Release** herunterladen: https://github.com/haroldlinke/WebMLL/releases (dort beim obersten Eintrag unter „Assets“ auf `webmobaledlib.zip` klicken). Direkter Download der neuesten Version: https://github.com/haroldlinke/WebMLL/releases/latest/download/webmobaledlib.zip
2. Einen **neuen Ordner** **`mobaledlib`** anlegen (zum Beispiel im eigenen Benutzerordner), die ZIP-Datei dort hineinkopieren und **entpacken**: Windows Rechtsklick → *Alle extrahieren* (im vorgeschlagenen Zielpfad ein angehängtes `\webmobaledlib` löschen), Linux und Mac „Hier entpacken“. Danach liegt der Ordner `python` direkt in `mobaledlib`.
3. Die **Komplettinstallation** mit einem Befehl starten (Tabelle unten). Sie richtet alles ein: die Python-Umgebung, die richtige **ARDUINO IDE 1.8.19** (nicht 2.x), die Bibliotheken und ein Symbol zum Starten. Das Fenster zeigt den Fortschritt; es dauert einige Minuten.
4. **Starten:** Symbol **„pyMobaLedLib“** (Windows: „pyMobaLedLib - Web“) doppelklicken. Nach 20 bis 60 Sekunden öffnet sich der Browser mit dem Programm.

System	Komplettinstallation
Windows 10/11	Ordner `mobaledlib\python\install` öffnen, oben in die Adressleiste des Explorers `cmd` tippen und Enter drücken. Dort eingeben: `install_windows.bat /ide`. Windows fragt nach der Erlaubnis. Wird dabei Python neu installiert, **den PC neu starten** und den Befehl noch einmal eingeben.
Linux (Mint, Debian, Ubuntu)	Terminal: `cd ~/mobaledlib/python/install` und dann `bash install_linux.sh --with-arduino-ide`. Danach einmal ab- und wieder anmelden.
macOS (ab 10.13, noch wenig getestet)	Terminal: `cd ~/mobaledlib/python/install` und dann `bash install_mac.command --with-arduino-ide`.
Raspberry Pi	siehe unten

Raspberry Pi: Besonderheiten und einfachster Weg

Der Pi läuft als **Dienst im Dauerbetrieb**: Das Programm startet beim Hochfahren und ist von jedem Rechner im Netz bedienbar (auch ohne Bildschirm am Pi).

1. Mit dem *Raspberry Pi Imager* das neue Betriebssystem **Raspberry Pi OS „Trixie“, 64-Bit, mit Desktop** auf die SD-Karte schreiben. Im Imager Benutzername, Passwort, WLAN und „SSH aktivieren“ einstellen. Pi starten.
2. Auf dem Pi im Browser die Datei `webmobaledlib.zip` herunterladen (derselbe Link wie oben), im Dateimanager einen **neuen Ordner** **`mobaledlib`** anlegen, die ZIP-Datei hineinkopieren und **„Hier entpacken“**.

3. Terminal öffnen und die Komplettinstallation starten:

```
cd ~/mobaledlib/python/pi
sudo bash install_pi.sh --with-arduino-ide
```

4. Danach: Desktop-Symbol **„pyMobaLedLib“** (öffnet das Programm auf dem Pi). Von einem anderen Rechner im Netz: `http://raspberrypi.local:8080` (statt `raspberrypi` der im Imager gewählte Rechnername), Benutzer `mobaledlib`, Passwort `mobaledlib` (im Programm unter „Programm Einstellungen“ änderbar). Der Pi wird über das Menü **Datei → Raspberry Pi herunterfahren** ausgeschaltet, nicht durch Ziehen des Steckers.

Mehr dazu, Optionen und die Fehlersuche stehen in den folgenden Kapiteln.

1. Welches System, welches Skript?

System	Kapitel	Installationsskript (im Projektordner)	Läuft als	Starten
Windows 10 / 11	3	`install\install_windows.bat`	Programm im Konsolenfenster	Verknüpfung „pyMobaLedLib - Web“ auf dem Desktop
macOS 10.13 oder neuer	4	`install/install_mac.command`	App `pyMobaLedLib.app`	Programm auf dem Schreibtisch
Linux-Desktop (Debian, Linux Mint, Ubuntu, Fedora, Arch …)	5	`install/install_linux.sh`	Programm im Benutzerkonto	Symbol im Anwendungsmenü und auf dem Schreibtisch
Raspberry Pi (Dauerbetrieb, auch ohne Bildschirm)	6	`pi/install_pi.sh`	Systemdienst, startet beim Hochfahren	Desktop-Symbol, oder Browser von einem anderen Rechner

Gemeinsam ist allen: Der Ordner **`install`** (und für Linux und Pi der Ordner **`pi`**) liegt **im Projektordner, neben** **`webmobaledlib.py`**. Die Skripte bestimmen den Projektordner selbst als den Ordner über ihrem eigenen. Es muss kein Pfad eingetragen werden.

2. Gemeinsame Grundlagen

2.1 Dateien und Ordnerstruktur

Der **gesamte Projektordner** **`python`** (mit `pyMobaLedLib.py`, `webmobaledlib.py` und allen anderen Programmdateien) liegt auf dem Rechner, auf dem das Programm läuft. Er enthält die Unterordner `install` (Skripte für Windows, macOS und Linux) und `pi` (Skripte für den Raspberry Pi; das Skript für die Bibliotheken wird auch unter Linux benutzt). Nicht nötig: `__pycache__`. (Konfiguration, Sicherungen und Protokolle liegen nicht im Projektordner, siehe 2.6.)

Das Programm wird mit **`webmobaledlib.py`** gestartet (das erledigen die Starter-Symbole). Im Projektordner liegen außerdem die Programmteile der Web-Oberfläche in den Ordnern **`headless`** und **`_web`**, die angepassten Originaldateien in `ExcelAPI` und `mlpyproggen` sowie die Skripte in `install` und `pi`.

Beispiele für die Lage des Projektordners: `D:\Development\github\pyMobaLedLib\python` (Windows),

`~/mobaledlib/python` (Linux, Pi). Die Skripte legen im Projektordner die Python-Umgebung `venv_web` bzw. `venv` an.

2.2 Die ARDUINO IDE: Version 1.8.19, nicht 2.x

Das Programm benutzt die **ARDUINO IDE 1.8.19** zum Kompilieren und Hochladen. Die neue **IDE 2.x wird nicht unterstützt.** Sie steht auf der Startseite von arduino.cc und wird deshalb oft aus Versehen installiert. Eine vorhandene IDE 2.x stört nicht, es wird zusätzlich die Version 1.8.19 gebraucht.

Am einfachsten installiert sie das Installationsskript mit der passenden Option:

System	Option	Ziel
Windows	`install_windows.bat /ide`	`C:\Program Files (x86)\Arduino` (Installer von arduino.cc, Rückfrage der Benutzerkontensteuerung)
macOS	`bash install_mac.command --with-arduino-ide`	`/Applications/Arduino.app` (ohne Schreibrecht `~/Applications`)
Linux-Desktop	`bash install_linux.sh --with-arduino-ide`	`~/arduino-1.8.19` (ohne root)
Raspberry Pi	`sudo bash install_pi.sh --with-arduino-ide`	`/opt/arduino-1.8.19` (Version für 64-Bit-ARM)

Die Option prüft zuerst, ob die IDE 1.8 schon vorhanden ist, und lädt dann nichts. Danach werden auch die Bibliotheken installiert (Abschnitt 2.3).

Von Hand: Auf arduino.cc unter *Software* im Abschnitt **„Legacy IDE (1.8.x)“** die Version für das System laden (Windows Installer, macOS, Linux 64 bit bzw. ARM 64 bit).

Wie das Programm die IDE findet: Es braucht die **Startdatei der IDE 1.8.x**, unter Linux und macOS die Datei `arduino`, unter Windows `arduino.exe`. Die IDE 2.x und die Version 1.0.5 aus der Paketverwaltung (`apt install arduino`) funktionieren nicht.

- **Windows:** Das Programm sucht selbst in `C:\Program Files (x86)\Arduino` und `C:\Program Files\Arduino`.
- **macOS:** in `/Applications/Arduino.app` und `~/Applications/Arduino.app`.
- **Linux und Raspberry Pi:** in dieser Reihenfolge: ein gültiger Pfad in „ARDUINO Einstellungen“, die Umgebungsvariable `MLL_ARDUINO_EXE`, `/opt/arduino-1.8*/arduino`, `/opt/arduino*/arduino`, `/usr/local/arduino*/arduino`, `~/arduino-1.8*/arduino`, `~/arduino*/arduino`, **eine Ebene tiefer** in `~/*/arduino-1.8*/arduino` (zum Beispiel `~/Downloads/arduino-1.8.19/arduino`) und zuletzt der Suchpfad (`arduino` im Terminal). Von mehreren Treffern gilt der mit der höchsten Nummer. Die Treffer stehen im Protokoll. Wird nichts gefunden, steht dort „ARDUINO IDE nicht gefunden“ mit allen geprüften Orten. Ein Misserfolg wird nur 10 Sekunden gemerkt: Eine danach installierte IDE wird ohne Neustart gefunden.

Am sichersten: Die IDE 1.8.19 nach `~/arduino-1.8.19` entpacken (das macht `install_linux.sh --with-arduino-ide`). Dann findet das Programm sie ohne Einstellung.

Pfad selbst eintragen („ARDUINO Einstellungen“): Die IDE kann irgendwo liegen, dann trägt man den **vollständigen Pfad der Startdatei** ein, zum Beispiel `/home/name/Programme/arduino-1.8.19/arduino` (nicht den Ordner). Ein Dateiauswahlfenster gibt es dafür **nicht**: Die Oberfläche läuft im Browser, und der Browser sieht nur die Dateien seines eigenen Rechners, der Pfad muss aber auf dem Rechner stehen, auf dem das **Programm** läuft (beim Raspberry Pi also auf dem Pi, nicht auf dem PC). Dafür helfen drei Dinge:

- **Prüfung:** Unter dem Feld steht, ob die Datei vorhanden ist, welche Version sie hat und ob der Eintrag falsch ist. Bleibt das Feld leer, steht dort, was die automatische Suche gefunden hat.
- **Schaltfläche „IDE suchen“:** Sie durchsucht bis zu 12 Sekunden lang die üblichen Orte und den Home-Ordner (bis vier Ebenen tief, ohne versteckte Ordner), außerdem `/opt`, `/usr/local` und `/usr/share`. Gefunden wird, was wie eine IDE 1.x aussieht (neben der Datei `arduino` liegen die Ordner `hardware` und `lib`). Mit einem Klick auf einen Treffer wird der Pfad eingetragen, danach die Einstellungen speichern. Die IDE 2.x wird nicht gefunden (sie wird nicht unterstützt).
- **Von Hand im Terminal:** `which arduino` oder `find ~ -maxdepth 5 -type f -name arduino -perm -u+x` zeigt den Pfad. Er lässt sich markieren und einfügen. Im Dateimanager hilft auf vielen Systemen Rechtsklick, **Eigenschaften** oder **Pfad kopieren**.

Der Haken „Individuellen Pfad … verwenden“ ist nur unter Windows nötig. Auf Linux und macOS gilt der eingetragene Pfad immer, wenn die Datei existiert.

2.3 Bibliotheken, Boards und die Tabelle „Libraries“

Zum Kompilieren braucht das Programm die Bibliothek **MobaLedLib** im Sketchbook-Ordner der IDE (Windows `Dokumente\Arduino\libraries`, macOS `~/Documents/Arduino/libraries`, Linux und Pi `~/Arduino/libraries`) und weitere Bibliotheken in bestimmten Versionen, zum Beispiel FastLED. Das Programm prüft die Bibliotheken und Boards beim Start („Überprüfe Bibliotheken und Boards“) und installiert fehlende in der geforderten Version.

- **MobaLedLib** installiert das Installationsskript mit den Optionen aus 2.2, nachträglich mit `/libs` (Windows), `--with-arduino-libs` (macOS und Linux) bzw. `bash pi/install_arduino_libs.sh` (Pi). Oder in der IDE: *Sketch → Bibliothek einbinden → Bibliotheken verwalten …*, „MobaLedLib“ suchen, *Installieren*.
- **Warum zuerst FastLED:** Die IDE zieht fehlende Abhängigkeiten von MobaLedLib immer in der **neuesten** Version nach, das Programm verlangt aber bestimmte Versionen (zum Beispiel FastLED 3.9.12). Die Skripte installieren deshalb zuerst FastLED in der geforderten Version und danach MobaLedLib. Die Voreinstellung ist 3.9.12, änderbar mit `MLL_FASTLED_VERSION=x.y.z` (Pi und Linux: `MLL_FASTLED_VERSION=x.y.z bash install_arduino_libs.sh`).
- **Kontrolle in der Tabelle „Libraries“:** Sie zeigt die installierten Bibliotheken mit ihrer Version und die vom Programm geforderten Versionen. So öffnest du sie: Button **„Optionen“** in der Symbolleiste → Reiter **„Update“** → Button **„Status der Bibliotheken“**. Die Tabelle „Libraries“ wird dann angezeigt und erscheint wie jede andere Tabelle als Tabellenreiter am unteren Rand. (Die Einstellung „Programmgenerator versteckte Tabellen anzeigen“ in „Programm Einstellungen“ blendet alle ausgeblendeten Tabellen dauerhaft ein; für die Bibliotheken ist sie nicht nötig.) Über der Tabelle stehen die Knöpfe:
  1. **Check actual versions** liest die installierten Versionen neu ein und setzt in der Spalte „Select“ einen **Haken (✓)** bei jeder fehlenden oder falschen Bibliothek (zum Beispiel FastLED 3.10.x statt 3.9.12).
  2. **Install selected** installiert alle Zeilen mit Haken in der geforderten Version neu. Das Programm fragt vorher nach und zeigt den Verlauf im Fenster „Installation der Boards und Bibliotheken“.
  3. **Delete selected** löscht die Bibliotheken bzw. Boards der Zeilen mit Haken.

  Den Haken einer Zeile setzt oder entfernst du mit einem **Klick in die Zelle der Spalte „Select“**.
- **Boards** (zum Beispiel Raspberry Pi Pico) installiert das Programm über dieselbe Tabelle „Libraries“. Boards sind groß (mehrere hundert MB) und brauchen Zeit.
- **ESP32:** Das Programm verlangt das ESP32-Paket in der Version 1.0.4. Auf Windows

(Intel/AMD) und Intel-Macs ist das wie bei der Tk-Version. Auf dem Raspberry Pi und anderen 64-Bit-ARM-Rechnern (auch Macs mit Apple-Chip, nicht geprüft) fehlen für diese Version die Werkzeuge, siehe Kapitel 10.

2.4 Starten, Beenden, Speichern

- Die Oberfläche öffnet sich im Browser unter `http://localhost:8080`. Der Start dauert 20 bis 60 Sekunden. Ist sie schon gestartet, öffnet ein weiterer Doppelklick auf das Symbol nur den Browser.
- **Beenden:** Knopf **„Beenden“** in der Oberfläche (oben rechts). Unter Windows schließt sich dabei das Konsolenfenster. Auf dem Pi startet der Dienst danach von selbst neu.
- Es sollte immer nur **ein Browser** gleichzeitig mit der Oberfläche arbeiten.
- **Tk-Version und Web-Version nicht gleichzeitig benutzen:** Beide arbeiten mit denselben Konfigurations- und Tabellendateien.
- **Speichern:** Das Programm speichert beim Beenden („Speichern und beenden“) und zwischendurch automatisch in die Datei `.autosave.json` im Datenordner (2.6). Dateien aus dem Menü „Datei“ werden im Browser heruntergeladen bzw. hochgeladen. Eine PGF-Datei kannst du auch einfach auf die Tabelle ziehen.
- **Meldungen** stehen in der Datei `webmobaledlib.log` im Datenordner (2.6, Einstellungen siehe 2.7), außerdem `webmobaledlib-console.log` bei macOS und Linux und `journalctl` auf dem Pi. Welche Programmversion läuft, steht unter dem Titel der Oberfläche (zum Beispiel „webmobaledlib 1.16“) und in den Köpfen der Dateien („Datei-Version“).

2.5 Das Programmsymbol

Das Symbol ist das des originalen ProgrammGenerators (Ring aus bunten LEDs, Tabelle, Stift) mit einer kleinen blauen Weltkugel unten rechts, damit man die Web-Version vom Tk-Programm unterscheiden kann. Dieselbe Datei steht als Symbol im Browserreiter. `install/make_icons.py` erzeugt daraus `mobaledlib.png` (Verknüpfungen, Raspberry Pi), `favicon.png`, `mobaledlib.ico` (Windows) und `mobaledlib.icns` (macOS). Ohne Weltkugel: `python make_icons.py --plain`. Die fertigen Dateien liegen bei; die Installer erzeugen sie nur neu, wenn sie fehlen. `make_icons.py` braucht Pillow (`pip install Pillow`), die Web-Oberfläche selbst nicht. Die Vorlage `mobaledlib_source.png` ist nur 128 × 128 Pixel groß und wirkt vergrößert etwas weich; wer sie höher aufgelöst hat, ersetzt die Datei und startet `make_icons.py` noch einmal.

2.6 Wo liegen meine Daten? (Datenordner)

Alles, was das Programm **selbst erzeugt**, liegt **außerhalb des Projektordners** in einem Datenordner des Benutzers. Dadurch bleiben deine Daten erhalten, wenn bei einem Update der Ordner `python` ersetzt oder überschrieben wird.

System	Datenordner
Windows	`%APPDATA%\pyMobaLedLib` (das ist `C:\Users\<Name>\AppData\Roaming\pyMobaLedLib`)
macOS	`~/Library/Application Support/pyMobaLedLib`
Linux, Raspberry Pi	`~/.local/share/pyMobaLedLib` (beim Pi für den Benutzer, unter dem der Dienst läuft)

Einen anderen Ort legt die Umgebungsvariable **`MLL_DATA_DIR`** fest (zum Beispiel `D:\MobaLedDaten`). Der Ordner wird bei Bedarf angelegt. Wo er ist, zeigen das Fenster beim Start, das Protokoll, der Menüpunkt **Hilfe → Über …** und der Befehl `python userdata.py` im Projektordner.

Was dort liegt: die Konfigurationsdateien (Einstellungen, Farbtest, Palette), die Sicherung der

Tabellen (wird beim Beenden mit „Speichern“ geschrieben und beim Start geladen), die Zwischensicherung `.autosave.json`, die Passwort-Datei `web_password.json` (nur als Hash) und die Protokolle `webmobaledlib.log` und `webmobaledlib-console.log`, das Absturzprotokoll `webmobaledlib-crash.log` sowie der Unterordner `nicegui` (interner Speicher der Oberfläche).

Übernahme alter Dateien: Lagen Konfiguration, Tabellensicherung oder Passwort-Datei eines früheren Stands noch im Projektordner, kopiert das Programm sie beim ersten Start einmalig in den Datenordner. Die alten Dateien im Projektordner bleiben liegen und dürfen gelöscht werden.

Was im Projektordner bleibt: das Programm, die Vorlagen und Definitionen und die Python-Umgebung (`venv_web` bzw. `venv`). Wird bei einem Update der ganze Ordner gelöscht und neu entpackt, ist auch die Python-Umgebung weg: dann das Installationsskript noch einmal ausführen.

Sichern und Umziehen: den Datenordner kopieren. **Prüfen, dass nichts im Projektordner landet:** `python install/check_project_writes.py snapshot` vor dem Start, nach einer Sitzung `python install/check_project_writes.py compare` (zeigt neue, geänderte und gelöschte Dateien im Projektordner, mit `--parent` auch eine Ebene darüber).

2.7 Protokoll (Log)

Das Programm schreibt wie die Tk-Version ein Protokoll mit dem Standard-Logger von Python. Es enthält auch die Meldungen der Programmmodule. Die Datei **`webmobaledlib.log`** liegt im Datenordner (2.6). Ohne Angabe wird auf der Stufe **DEBUG** protokolliert, wie im Original. Die Datei dreht sich bei 2 MB (die älteren heißen `webmobaledlib.log.1` bis `.5`), und auf der Konsole (beim Pi im Journal des Dienstes) erscheinen die Meldungen ab INFO. Jede Sitzung beginnt mit einem Kopf aus Python-Version, System, Parametern, Ordnern, Stufe, Dateiname, den Versionen der wichtigsten Pakete und der Umgebung (Sitzungsart, Display, Tk). Außerdem steht darin, wann sich ein Browser verbindet und trennt (`Browser verbunden` / `Browser getrennt`), wie lange der Aufbau der Seite gedauert hat, und eine Warnung, wenn der Server ungewöhnlich lange blockiert war. Bricht der Python-Prozess ab (zum Beispiel durch einen Fehler in einer Systembibliothek), steht der Grund in `webmobaledlib-crash.log`. In der Oberfläche zeigt **Hilfe → Logfile anzeigen** die letzten Zeilen (am Ende steht das Neueste) mit Pfad, Größe und Stufe der Datei. Mit **Aktualisieren** liest es neu, mit **Ganze Datei herunterladen** bekommst du die komplette Datei, zum Beispiel für eine Fehlermeldung.

Kommandozeile	Umgebungsvariable	Wirkung
`--loglevel DEBUG`, `INFO`, `WARNING`, `ERROR` oder `CRITICAL`	`MLL_LOGLEVEL`	Stufe der Protokolldatei (Standard `DEBUG`)
`--logfile NAME`	`MLL_LOGFILE`	Dateiname. Ohne Pfad liegt die Datei im Datenordner, mit absolutem Pfad dort, `stdout` schreibt keine Datei

Beispiele: Windows `install\start_windows.bat --loglevel INFO`, Linux und macOS `install/start_linux.sh --loglevel INFO`, Raspberry Pi die Zeile `MLL_LOGLEVEL=INFO` in `/etc/mobaledlib-web.env` und danach `sudo systemctl restart mobaledlib-web`. Die sehr gesprächigen Bibliotheken (Webserver, Websocket) bleiben auch bei DEBUG gedämpft. Wer sie sehen will, setzt `MLL_LOG_LIBS=1`. Bei einer Fehlermeldung bitte die Datei `webmobaledlib.log` schicken (und `webmobaledlib.log.1`, falls vorhanden).

2.8 Kleine Bildschirme: kompaktes Layout

Auf kleinen Bildschirmen (zum Beispiel einem Laptop mit 1366 × 768 Punkten, auf dem der Browser nur rund 600 Punkte Höhe hat) bleibt für die Tabelle wenig Platz, weil Kopfzeile, Menü, Reiter und die großen Symbole der Funktionsknöpfe mehr als die Hälfte belegen. Das **kompakte**

Layout schafft Platz:

- Menü (Datei, Tabelle, ARDUINO, Hilfe), Titel, Statusmeldung und der Knopf **Beenden** stehen in **einer** Zeile. Der Titel ist kleiner und ohne „(Web, Prototyp)“, der Knopf Beenden ohne Symbol, die Versionsangabe und die Zeit der Zwischensicherung sind ausgeblendet (die Version steht unter Hilfe → Über …).
- Die Reiter sind schmaler.
- Die **Symbole der Funktionsknöpfe werden durch ihren Text ersetzt:** eine schmale Reihe kleiner Textknöpfe statt großer Symbole.
- Leerzeilen, Ränder und Abstände entfallen.
- In den **Makrodialogen** (Makroauswahl, Parameterformulare, Haus-Dialog) sind die **Beschreibungstexte auf eine Zeile eingeklappt**. Ein Klick auf den Text (Pfeil ▸) klappt ihn auf, ein weiterer Klick wieder zu.
- Die Zeile „Zeile x letzte benutzte Zeile von y Zeilen“ steht unten rechts neben den Blattreitern statt in einer eigenen Zeile.

Einschalten: Reiter **Programm Einstellungen**, Karte **Darstellung auf kleinen Bildschirmen**, Feld **Kompaktes Layout**:

Stufe	Wirkung
Aus (Standard)	normales Layout
Ein	immer kompakt
Automatisch	kompakt, wenn das Browserfenster niedriger als 800 Pixel ist, und wieder normal, wenn es höher wird

Die Auswahl wirkt sofort, gesichert wird sie mit **Speichern**. Die Einstellung gilt für das Programm auf diesem Rechner (sie steht in der Konfiguration im Datenordner). Ohne gespeicherten Wert gilt die Umgebungsvariable `MLL_LAYOUT` (`normal`, `compact` oder `auto`). Zusätzlich schaffen **Vollbild (F11)** und der **Browser-Zoom** (Strg und Minus) Platz.

3. Windows

Voraussetzungen: Windows 10 oder 11, Python 3.10 oder neuer (das Skript bietet die Installation an), Internet für die Installation der Python-Pakete.

Installation

1. Im Ordner `install` **`install_windows.bat` doppelklicken**. Für alles in einem Zug (IDE und Bibliotheken) in einer Eingabeaufforderung im Ordner `install` `install_windows.bat /ide` eingeben. Meldet Windows „Der Computer wurde durch Windows geschützt“: *Weitere Informationen → Trotzdem ausführen*.
2. Das Skript sucht Python 3.10 oder neuer. Fehlt es, bietet es die Installation von Python 3.12 mit `winget` an und sucht danach in den Standardordnern weiter, die Installation läuft dann ohne Pause weiter. Findet es Python danach nicht, meldet es das: **Dann den PC neu starten** (oder ab- und wieder anmelden) und das Skript noch einmal starten. Erst nach dem Neustart kennt Windows den neuen Suchpfad von Python.
3. Es legt die Python-Umgebung `venv_web` im Projektordner an, installiert die Pakete aus `requirements-web.txt`, erzeugt das Programmsymbol und legt auf dem Desktop die Verknüpfung **„pyMobaLedLib - Web“** an.
4. Mit `/ide` lädt es die IDE 1.8.19 (ca. 130 MB) von arduino.cc und führt den Installer still aus. Windows fragt nach der Erlaubnis (Benutzerkontensteuerung); den Treiber für den USB-Anschluss bitte zulassen. Danach installiert es FastLED und MobaLedLib. (Nur die Bibliotheken, wenn die IDE schon da ist: `/libs`.)
5. Am Ende läuft ein Test der Installation (`pi_check.py`, etwa eine Minute). Ohne ihn: `/nocheck`.

Option	Wirkung
`/ide`	ARDUINO IDE 1.8.19 laden und installieren, dazu `/libs`
`/libs`	Bibliotheken FastLED (3.9.12) und MobaLedLib mit `arduino_debug.exe` installieren
`/nocheck`	ohne den Test der Installation am Ende

Starten: Verknüpfung auf dem Desktop doppelklicken (oder `install\start_windows.bat`). Es öffnet sich ein (minimiertes) Konsolenfenster und nach 20 bis 60 Sekunden der Browser. Das Konsolenfenster nicht schließen. **Beenden:** Knopf „Beenden“ in der Oberfläche.

Besonderheiten: Der Pfad des Projektordners sollte kein `'` und kein `&` enthalten (sonst kann die Verknüpfung nicht angelegt werden; dann `install\start_windows.bat` per Rechtsklick → *Senden an → Desktop*). Die Windows-Firewall fragt nicht, weil die Oberfläche nur auf dem eigenen Rechner lauscht.

4. macOS

Voraussetzungen: macOS 10.13 oder neuer, Python 3.10 oder neuer (das von macOS mitgelieferte Python ist zu alt), Internet. **macOS 10.11 und 10.12 gehen nicht:** Für diese Systeme gibt es keine fertigen Pakete (zum Beispiel `orjson`), die Installation der Python-Pakete scheitert.

Installation

1. Im Ordner `install` **`install_mac.command` doppelklicken**. Ist es nicht ausführbar, im Terminal `chmod +x install/install_mac.command` und dann `bash install/install_mac.command`. Für alles in einem Zug: `bash install_mac.command --with-arduino-ide`.
2. Das Skript sucht Python 3.10 oder neuer und bietet bei fehlendem Python die Installation mit Homebrew an. Sonst: Python 3.12 von python.org installieren und das Skript noch einmal starten.
3. Es legt `venv_web` im Projektordner an, installiert die Pakete, erzeugt das Symbol und legt auf dem Schreibtisch das Programm **`pyMobaLedLib.app`** an.
4. Mit `--with-arduino-ide` lädt es `arduino-1.8.19-macosx.zip` (ca. 200 MB) und kopiert `Arduino.app` nach *Programme*, danach installiert es FastLED und MobaLedLib. Auf Macs mit Apple-Chip fragt macOS beim ersten Start der IDE nach Rosetta: bitte zustimmen. (Nur die Bibliotheken: `--with-arduino-libs`.)
5. Am Ende läuft ein Test der Installation (ohne ihn: `--nocheck`).

Option	Wirkung
`--with-arduino-ide`	ARDUINO IDE 1.8.19 laden und installieren, dazu `--with-arduino-libs`
`--with-arduino-libs`	Bibliotheken FastLED (3.9.12) und MobaLedLib mit der IDE installieren
`--nocheck`	ohne den Test der Installation am Ende

Starten: `pyMobaLedLib` auf dem Schreibtisch doppelklicken. Beim ersten Mal fragt macOS eventuell: Rechtsklick → *Öffnen* → *Öffnen*. Die Oberfläche startet ohne Terminalfenster und öffnet den Browser. Meldungen: `webmobaledlib-console.log` und `webmobaledlib.log` im Datenordner (2.6). Zum Beobachten im Terminal: `install/start_mac.command`. **Beenden:** Knopf „Beenden“ in der Oberfläche.

5. Linux-Desktop (Debian, Linux Mint, Ubuntu, Fedora, Arch …)

Das Programm läuft im Benutzerkonto, ohne Systemdienst. Du startest es mit einem Symbol, die Oberfläche öffnet sich im Browser.

Voraussetzungen: Debian 12, Linux Mint 21 / 22, Ubuntu 22.04 oder neuer (Fedora, Arch, openSUSE gehen auch, die Paketnamen setzt das Skript). **Python 3.10 oder neuer** (Mint 20 und Debian 11 sind zu alt). Ein Benutzer mit `sudo` (nur für die Systempakete und die Gruppe `dialout`). Internet.

Installation (als normaler Benutzer, **ohne** sudo):

```
cd ~/mobaledlib/python/install
bash install_linux.sh --with-arduino-ide
```

Das Skript erledigt der Reihe nach:

1. Systempakete (`python3`, `python3-venv`, `python3-pip`, `curl`, `xz-utils`), nur was fehlt, mit einer Passwortabfrage von `sudo`.
2. Prüfung auf Python 3.10 oder neuer.
3. Die Python-Umgebung `venv_web` im Projektordner mit den Paketen aus `requirements-web.txt`.
4. Der Benutzer kommt in die Gruppe **`dialout`** (Zugriff auf `/dev/ttyUSB0`, `/dev/ttyACM0`). **Danach einmal ab- und wieder anmelden.**
5. Mit `--with-arduino-ide`: die IDE 1.8.19 nach `~/arduino-1.8.19` (etwa 150 MB) und danach FastLED und MobaLedLib.
6. Ein Eintrag im Anwendungsmenü (Entwicklung / Elektronik) und ein Symbol auf dem Schreibtisch.
7. Ein Test der Installation (`pi_check.py`, etwa eine Minute).

Option	Wirkung
`--with-arduino-ide`	IDE 1.8.19 laden, dazu `--with-arduino-libs`
`--with-arduino-libs`	nur die Bibliotheken (die IDE ist schon da)
`--no-desktop-icon`	kein Symbol auf dem Schreibtisch
`--no-dialout`	die Gruppe `dialout` nicht eintragen
`--no-check`	ohne den Test der Installation

Das Skript kann **mehrfach** laufen (zum Beispiel nach einem Update).

Starten: Symbol „pyMobaLedLib“ doppelklicken (Schreibtisch oder Anwendungsmenü), oder `install/start_linux.sh`. Beim ersten Doppelklick fragt der Dateimanager (Nemo, Nautilus, Dolphin) eventuell **„Ausführen“** bzw. **„Starter vertrauen“**; sonst Rechtsklick → *Ausführen erlauben* (Mint: in den Eigenschaften *Als Programm ausführen*). **Beenden:** Knopf „Beenden“ in der Oberfläche. Zum Beobachten der Meldungen `install/start_linux.sh` im Terminal starten. Auf dem Desktop läuft die IDE mit deinem Bildschirm, es wird kein virtuelles Display gebraucht.

6. Raspberry Pi (Dauerdienst)

Der Programmgenerator läuft als Dienst auf dem Pi. Du startest ihn mit einem **Desktop-Symbol** oder öffnest ihn mit dem Browser, auf dem Pi selbst oder von jedem Rechner im Netz. Der Arduino/Pico steckt per USB am Pi. Es wird kein Tk gebraucht.

Voraussetzungen: Raspberry Pi 4 oder 5 (ein Pi 3 läuft, Kompilieren dauert dort deutlich länger); Raspberry Pi OS **64 Bit**, **Trixie** (Debian 13, Python 3.13) oder Bookworm, mit Desktop für das Symbol (Lite geht auch, dann ohne Symbol); 2 GB RAM sollten reichen, 4 GB sind sicherer; ein Benutzer mit `sudo`-Recht (im Folgenden `pi`, bei dir kann er anders heißen); Internet für die Installation.

6.1 Dateien auf den Pi bringen

Der **gesamte Projektordner** `python` kommt auf den Pi, so wie er auf dem PC läuft. Auf dem Pi liegt er **in** `~/mobaledlib`:

```
/home/pi/mobaledlib/python/                 <- Projektordner (hier liegt webmobaledlib.py)
/home/pi/mobaledlib/python/pi/              <- Installationsskripte (install_pi.sh ...)
```

So entsteht es mit einer ZIP-Datei:

1. Auf dem PC den **Ordner** `python` **selbst** als ZIP packen (nicht nur seinen Inhalt), ohne `__pycache__`.
2. Die ZIP-Datei auf den Pi in den Ordner `~/mobaledlib` kopieren (`mkdir -p ~/mobaledlib`, falls es ihn nicht gibt).
3. Dort **„Hier entpacken“**: Es entsteht automatisch der Ordner `python` mit allen Dateien darunter. (Enthält die ZIP nur den Inhalt ohne Ordner, würden die Dateien lose in `~/mobaledlib` landen: dann mit „Entpacken nach …“ in einen neuen Ordner `python` entpacken.)

Alternativ im Terminal:

```
        # vom Windows-PC (PowerShell); zuvor auf dem Pi:  mkdir -p ~/mobaledlib
        scp -r D:\Development\github\pyMobaLedLib\python pi@raspberrypi.local:/home/pi/mobaledlib/
        # oder auf dem Pi:  git clone <dein Repository> ~/mobaledlib       (das Repository enthält den
r python)
```

6.2 Installation (ein Befehl)

```
        cd /home/pi/mobaledlib/python/pi
        sudo bash install_pi.sh --with-arduino-ide
```

Das Skript erledigt der Reihe nach:

1. Systempakete (Python, venv, `xvfb`, ein paar Bibliotheken für die Java-IDE),
2. eine eigene Python-Umgebung `venv` mit den Paketen aus `requirements-pi.txt`,
3. Zugriff auf die USB-Anschlüsse (Gruppe `dialout`),
4. `/etc/mobaledlib-web.env` mit Port und Passwort (Voreinstellung: Benutzer `mobaledlib`, Passwort `mobaledlib`, im Programm änderbar, siehe Kapitel 8),
5. den Starter für das Desktop-Symbol und eine sudo-Regel, mit der du **nur** den Dienst `mobaledlib-web` ohne Passwort starten, anhalten und neu starten sowie den Pi herunterfahren und neu starten darfst. Dazu kommt ein kleines Hilfsprogramm (`/usr/local/sbin/mobaledlib-`

`set-hostname`, Besitzer root), mit dem die Oberfläche den Rechnernamen ändern darf (6.7). Das Hilfsprogramm prüft den Namen streng,

6. das **Desktop-Symbol** (6.3),
7. die ARDUINO IDE 1.8.19 (nur mit `--with-arduino-ide`, etwa 150 MB Download) und die Bibliothek MobaLedLib mit ihren Abhängigkeiten,
8. den Dienst `mobaledlib-web` (startet beim Hochfahren) und einen Test der Installation (`pi_check.py`).

Option	Wirkung
`--with-arduino-ide`	IDE 1.8.19 laden, dazu `--with-arduino-libs`
`--with-arduino-libs`	nur die Bibliotheken
`--port 9000`	anderer Port (Voreinstellung 8080)
`--hostname webMLLraspi`	Rechnernamen des Pi ändern, danach erreichbar als `http://webMLLraspi.local:8080` (6.7)
`--password GEHEIM`	anderes Anfangspasswort festlegen (Voreinstellung: `mobaledlib`)
`--local-only`	nur auf dem Pi erreichbar, kein Passwort
`--desktop-icon` / `--no-desktop-icon`	Symbol auch ohne erkannten Desktop / kein Symbol
`--no-check`	ohne den Test der Installation

Das Skript kann **mehrfach** laufen (zum Beispiel nach einem Update). Vorhandene Einstellungen bleiben erhalten. Windows-Zeilenenden (CRLF) in den Skripten entfernt das Skript selbst. Falls schon der Start mit `$'\r': command not found` abbricht: `sed -i 's/\r$//' install_pi.sh` und noch einmal starten.

6.3 Das Desktop-Symbol

Nach der Installation liegt auf dem Desktop das Symbol **„pyMobaLedLib“** (und ein Eintrag im Startmenü unter Entwicklung / Elektronik). Ein Doppelklick:

1. startet den Dienst, wenn er nicht läuft (nach einem „Beenden“ im Programm zum Beispiel),
2. wartet, bis die Oberfläche bereit ist (20 bis 60 Sekunden, dabei erscheint die Meldung „pyMobaLedLib wird gestartet“),
3. öffnet die Oberfläche in einem **eigenen Chromium-Fenster** ohne Adressleiste. Fehlt Chromium, nimmt er Firefox, sonst den Standardbrowser. Mit `MLL_BROWSER=<Programm>` wählst du einen anderen.

Auf dem Pi selbst brauchst du **kein Passwort** (Zugriffe von `localhost`). Aus dem Netz gilt es weiter. Fragt der Desktop beim ersten Doppelklick, ob die Datei ausgeführt werden soll, wähle „Ausführen“. Kommt nichts, starte den Starter im Terminal, dann siehst du die Meldung: `/home/pi/mobaledlib/python/pi/start_mobaledlib.sh`.

6.4 ARDUINO IDE und Bibliotheken auf dem Pi

Die allgemeinen Hinweise stehen in 2.2 und 2.3. Besonderheiten des Pi:

- Mit `--with-arduino-ide` lädt das Skript die IDE 1.8.19 für 64-Bit-ARM (`linuxaarch64`) und packt sie nach `/opt/arduino-1.8.19`. Von Hand: die Datei `arduino-1.8.19-linuxaarch64.tar.xz` laden, nach `/opt` auspacken.
- Die Bibliothek MobaLedLib installiert das Skript **ohne sudo** als der Benutzer, unter dem der Dienst läuft (sie liegt in `~/Arduino/libraries/MobaLedLib`):

```
bash ~/mobaledlib/python/pi/install_arduino_libs.sh
sudo systemctl restart mobaledlib-web
```

- Boards, gleich mit Version:

```
bash ~/mobaledlib/python/pi/install_arduino_libs.sh --pico            # Raspberry Pi Pico
0)
bash ~/mobaledlib/python/pi/install_arduino_libs.sh --pico=5.4.2 --esp32
```

  (Das ESP32-Paket lässt sich auf dem Pi nicht installieren, siehe Kapitel 10.)

6.5 Alltag auf dem Pi

Was	Befehl
Starten	Desktop-Symbol (oder `sudo systemctl start mobaledlib-web`)
Läuft der Dienst?	`systemctl status mobaledlib-web`
Meldungen ansehen	`journalctl -u mobaledlib-web -f` (und die Datei `webmobaledlib.log` im Datenordner, 2.6)
Neu starten / anhalten	`sudo systemctl restart mobaledlib-web` / `sudo systemctl stop mobaledlib-web`
Nicht mehr beim Hochfahren starten	`sudo systemctl disable mobaledlib-web`
Installation prüfen	`cd ~/mobaledlib/python && MLL_TK_ROOT=0 venv/bin/python pi_check.py`
Herunterfahren / Neustart	Menü **Datei → Raspberry Pi herunterfahren …** (bzw. **neu starten …**), siehe 6.6
Alles wieder entfernen	`sudo bash ~/mobaledlib/python/pi/uninstall_pi.sh` (Projektordner, venv und IDE bleiben)

Der Knopf **„Beenden“** in der Oberfläche beendet das Programm. Der Dienst startet danach von selbst neu; mit dem Symbol öffnest du die Oberfläche dann wieder. Soll er aus bleiben: `sudo systemctl stop mobaledlib-web`.

6.6 Raspberry Pi herunterfahren

Ein Pi sollte nicht einfach vom Strom getrennt werden: Dabei kann die SD-Karte beschädigt werden. Die Oberfläche hat deshalb im Menü **Datei** die Einträge **Raspberry Pi herunterfahren …** und **Raspberry Pi neu starten …** (nur auf dem Pi sichtbar).

1. Eine Rückfrage fragt: **Speichern und herunterfahren**, **Ohne Speichern** oder Abbrechen.
2. Das Programm wird wie beim Knopf „Beenden“ ordentlich abgeschlossen (Tabellen und Konfiguration sichern, Arduino trennen).
3. Es erscheint die Seite „Der Raspberry Pi wird heruntergefahren“. Danach läuft `sudo systemctl poweroff`.
4. **Warte, bis die grüne LED am Pi nicht mehr blinkt** (etwa 10 Sekunden), dann kannst du den Strom trennen.

Dafür muss der Dienstbenutzer `systemctl poweroff` und `systemctl reboot` ohne Passwort ausführen dürfen. Das legt `install_pi.sh` an (sudo-Regel) und trägt `MLL_ALLOW_SHUTDOWN=1` in `/etc/mobaledlib-`

`web.env` ein. Bei einer älteren Installation einmal `sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` ausführen. Fehlt die Regel, sagt die Oberfläche das und beendet das Programm nicht. Im Terminal geht es auch mit `sudo shutdown -h now`.

Mit einem Taster am Pi (optional): Ein Taster zwischen GPIO 3 (Pin 5) und GND (Pin 6) kann den Pi herunterfahren und wieder starten. In `/boot/firmware/config.txt` die Zeile `dtoverlay=gpio-shutdown` ergänzen und neu starten. Das Programm wird dabei nicht gefragt: Der Dienst wird von systemd beendet, nicht gespeichert. Vorher in der Oberfläche speichern.

6.7 Den Raspberry Pi im Netz finden

Von einem anderen Rechner brauchst du die Adresse des Pi. Die einfachsten Wege, in dieser Reihenfolge:

1. **Name im Netz (mDNS): `http://raspberrypi.local:8080`.** Der Pi meldet sich mit dem Dienst `avahi-daemon` unter `<Rechnername>.local` im Netz (auf Raspberry Pi OS vorhanden, das Installationsskript installiert ihn, falls er fehlt). Das funktioniert unter Windows 10 (neuere Versionen) und 11, macOS, iPhone und iPad und unter Linux mit `avahi` bzw. `libnss-mdns` (bei Linux Mint und Ubuntu Standard). Auf Android klappt es oft nicht. Der Rechnername ist der, den du im Raspberry Pi Imager gewählt hast (sonst `raspberrypi`). Test: `ping raspberrypi.local`.
2. **Eigener, kurzer Name** (zum Beispiel `webMLLraspi`; Groß- und Kleinschreibung spielt bei der Suche keine Rolle, erlaubt sind Buchstaben, Ziffern und `-`, höchstens 31 Zeichen). Es gibt drei Wege:
   - **Im Programm: Programm Einstellungen** → Block **„Name des Raspberry Pi im Netz“** → neuen Namen eingeben → **„Namen ändern“** und die Rückfrage bestätigen. Der Name gilt sofort, ohne Neustart. Die Oberfläche zeigt danach die neue Adresse an. Die alte Adresse gilt dann nicht mehr: Lesezeichen und das Desktop-Symbol (es benutzt `localhost` und ist nicht betroffen) entsprechend anpassen. Den Block gibt es nur auf dem Pi und nur, wenn die Installation mit `install_pi.sh` ab Version 1.06 gelaufen ist (sie legt das Hilfsprogramm und die sudo-Regel an).
   - **Bei der Installation:** `sudo bash install_pi.sh --no-check --hostname webMLLraspi`.
   - **Von Hand:** `sudo raspi-config nonint do_hostname webMLLraspi` und neu starten.
3. **Der Router:** Dort steht der Pi in der Liste der Geräte, bei der FRITZ!Box unter *Heimnetz → Netzwerk → Netzwerkverbindungen* mit seiner IP-Adresse. Die FRITZ!Box löst auch Namen auf: `http://raspberrypi.fritz.box:8080`. Empfehlung: dem Pi im Router eine **feste IP-Adresse** geben (FRITZ!Box: Gerät bearbeiten → „Diesem Netzwerkgerät immer die gleiche IPv4-Adresse zuweisen“), dann ändert sie sich nicht.
4. **Suchprogramm auf dem PC:** `install/find_mobaledlib.py` probiert zuerst `raspberrypi.local` und `mobaledlib.local` und fragt dann alle Adressen des eigenen Netzes (/24) auf dem Port 8080 ab. Die Oberfläche erkennt es an der Anmeldung „pyMobaLedLib“ bzw. dem Seitentitel und zeigt Adresse und Namen. Aufruf: `python install/find_mobaledlib.py` (Windows nach der Installation: `venv_web\Scripts\python install\find_mobaledlib.py`). Andere Netze oder Ports: `--subnet 192.168.178.0/24`, `--port 9000`, `--name mein-pi`.
5. **Am Pi selbst:** `hostname -I` zeigt die IP-Adresse, `hostname` den Namen. Beide stehen auch am Ende der Installation.

7. Aktualisieren und Deinstallieren

System	Aktualisieren	Deinstallieren
alle	Menü **Hilfe → Update webmobaledlib …** (empfohlen, siehe 7.1), oder neue Programmdateien in den Projektordner kopieren oder den Ordner `python` ersetzen (die Version steht im Dateikopf und unter dem Titel der Oberfläche), Oberfläche neu starten. **Das laufende Programm vorher beenden** (Knopf BEENDEN oben rechts): Läuft die alte Instanz noch, öffnet der Starter nur den Browser auf ihr, und es erscheint die alte Version, obwohl die Dateien neu sind. Deine Daten liegen im Datenordner und bleiben erhalten (2.6)	–
Windows	Neue Pakete: `venv_web\Scripts\pip install -U -r install\requirements-web.txt`, oder das Skript noch einmal ausführen	Verknüpfung und Ordner `venv_web` löschen
macOS	`venv_web/bin/pip install -U -r install/requirements-web.txt`, oder das Skript noch einmal ausführen	`pyMobaLedLib.app` und Ordner `venv_web` löschen
Linux-Desktop	`venv_web/bin/pip install -U -r install/requirements-web.txt`, oder `install_linux.sh` noch einmal	`bash install/uninstall_linux.sh` (mit `--purge` auch `venv_web`)
Raspberry Pi	`sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` (aktualisiert auch die Python-Pakete und die Dienstdatei)	`sudo bash ~/mobaledlib/python/pi/uninstall_pi.sh`

Beim Deinstallieren bleiben Projektordner, ARDUINO IDE und Bibliotheken erhalten.

Sichern: den Datenordner (Konfiguration, Sicherungen der Tabellen, siehe 2.6) und deine Anlagendateien. Die Daten der Tabelle speicherst du mit dem Menü „Datei“ als Download im Browser (die Datei landet auf dem Rechner, an dem du sitzt). Auf dem Pi geht auch ein fertiges Abbild der SD-Karte (Raspberry Pi Imager oder `dd`).

7.1 Update über das Programm (Hilfe → Update webmobaledlib …)

Das Programm lädt das neueste Release (`webmobaledlib.zip` aus https://github.com/haroldlinke/WebMLL/releases), vergleicht **jede Datei** mit der installierten und zeigt, was sich ändert: neue, geänderte und zu löschende Dateien, mit Versionsnummer. Erst nach **Aktualisieren** passiert etwas.

- **Nur Geändertes wird geschrieben.** Gleiche Dateien werden nicht angefasst. Das sind meist nur wenige Dateien statt des ganzen Ordners, und Virenscanner schlagen deshalb nicht an. Jede Datei wird zuerst in eine Hilfsdatei geschrieben und dann ersetzt.
- **Gelöscht werden nur Dateien, die ein früheres Update installiert hat** und die im neuen Release fehlen (die Liste dazu steht in `update_manifest.json` im Datenordner). Beim **ersten Update** einer Installation gibt es diese Liste noch nicht: Dann wird nichts gelöscht. Eigene Dateien im Projektordner bleiben immer erhalten.
- **Nie angefasst** werden die Python-Umgebung (`venv`, `venv_web`), `.git`, `__pycache__`, Protokolle, Konfiguration, Passwort-Datei und die Sicherungen der Tabellen, auch wenn ein ZIP solche Dateien enthalten sollte.
- **Sicherung:** Ersetzte und gelöschte Dateien liegen vorher im Datenordner unter

`update_backup/<Datum-Uhrzeit>/` (die letzten 5). Zum Zurückholen die Dateien in den Projektordner zurückkopieren. Die Tabellen werden vor dem Ersetzen gesichert.

- **Python-Pakete:** Haben sich die Anforderungen geändert (`install/requirements-web.txt`, auf dem Pi `pi/requirements-pi.txt`), aktualisiert das Programm sie.
- **Neustart:** Nach dem Update läuft noch die alte Version. **Neu starten** sichert die Tabellen und startet das Programm neu, der offene Reiter lädt sich selbst neu. Auf dem Raspberry Pi startet der Dienst von selbst neu. Haben sich Dateien im Ordner `pi` geändert, muss dort einmal `sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` laufen (die installierte Dienstdatei ist eine Kopie).
- **Entwicklungsordner:** In einer Git-Arbeitskopie ist das Update gesperrt, damit es die Dateien nicht mit dem Release überschreibt (`MLL_ALLOW_UPDATE=1` hebt die Sperre auf). `MLL_UPDATE_URL` richtet das Update auf eine andere ZIP-Adresse.
- **Sicherheit:** Es wird nur über HTTPS geladen. Das ZIP muss den Ordner `python` mit `webmobaledlib.py` und `_web` enthalten, Pfade nach außen und Verknüpfungen werden abgelehnt, und aus dem ZIP wird nichts ausgeführt.
- **Meldung „konnte nicht geladen werden“:** Internetverbindung, Firewall oder Proxy prüfen, und ob das Release die Datei `webmobaledlib.zip` enthält. „Nicht ersetzt“ bei einer Datei heißt meist, dass ein anderes Programm sie gesperrt hat: Programm beenden und das Update wiederholen.

8. Zugriff aus dem Netz und Passwort

Standardmäßig ist die Oberfläche **nur auf dem eigenen Rechner** erreichbar (`http://localhost:8080`). Auf dem **Raspberry Pi** ist sie im Netz erreichbar und mit einem Passwort geschützt. Beim Aufruf von einem anderen Rechner fragt der Browser nach Benutzername und Passwort.

Voreinstellung auf dem Pi: Benutzername **`mobaledlib`**, Passwort **`mobaledlib`**. Auf dem Pi selbst (Desktop-Symbol) ist kein Passwort nötig.

Passwort ändern (empfohlen, sobald der Pi im Netz erreichbar ist): In der Oberfläche **Programm Einstellungen** öffnen, im Block **„Zugriff über das Netz (Web-Oberfläche)“** das neue Passwort zweimal eingeben (mindestens 4 Zeichen) und **„Passwort ändern“** drücken. Es gilt sofort, ohne Neustart. Von einem anderen Rechner fragt der Browser beim nächsten Aufruf nach dem neuen Passwort. Solange noch das Standardpasswort gilt, zeigt der Block eine Warnung.

- Das neue Passwort steht nicht im Klartext auf dem Pi, sondern nur als Hash in der Datei `web_password.json` im Datenordner (2.6). Sie geht dem Passwort aus `/etc/mobaledlib-web.env` vor.
- **Passwort vergessen:** Entweder auf dem Pi in der Oberfläche **„Eigenes Passwort entfernen“** drücken (auf dem Pi selbst braucht man kein Passwort), oder die Datei `web_password.json` im Datenordner löschen. Dann gilt wieder das Passwort aus `/etc/mobaledlib-web.env` (Voreinstellung `mobaledlib`).
- Das Anfangspasswort lässt sich bei der Installation mit `--password` festlegen, oder später in `/etc/mobaledlib-web.env` (`MLL_PASSWORD=...`).

Auf anderen Systemen (Windows, macOS, Linux-Desktop) gibt es standardmäßig keinen Zugriff aus dem Netz und kein Passwort. Für den Zugriff aus dem Netz vor dem Start die Umgebungsvariablen `MLL_HOST=0.0.0.0` und ein Passwort `MLL_PASSWORD=...` setzen (Benutzername `MLL_USER`, Voreinstellung `mobaledlib`). Wo:

System	Wo
Windows	`set MLL_HOST=0.0.0.0` und `set MLL_PASSWORD=...` in `install\start_windows.bat`
macOS	`export ...` in der Datei `pyMobaLedLib.app/Contents/MacOS/pyMobaLedLib`
Linux-Desktop	`export ...` in `install/start_linux.sh`
Raspberry Pi	in `/etc/mobaledlib-web.env` (`sudo nano /etc/mobaledlib-web.env`), danach `sudo systemctl restart mobaledlib-web`

Die Datei auf dem Pi sieht so aus:

```
MLL_HOST=0.0.0.0        # 127.0.0.1 = nur auf dem Pi selbst
MLL_PORT=8080
MLL_USER=mobaledlib
MLL_PASSWORD=mobaledlib # Anfangspasswort; ohne diese Zeile ist die Oberfläche im Netz für jeden off
en
# MLL_LOCAL_NO_PASSWORD=0   # mit 0 gilt das Passwort auch auf dem Pi selbst
```

Nach einer Änderung des Ports einmal `sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` aufrufen, damit das Symbol den neuen Port kennt.

Sicherheit: Wer die Oberfläche erreicht, kann Programme auf den angeschlossenen Arduino laden (und auf dem Pi den Rechner herunterfahren). Das Passwort wird ohne Verschlüsselung (kein https) übertragen, und das Standardpasswort ist allgemein bekannt. Nutze den Zugriff nur im eigenen Heimnetz, ändere das Passwort und mach den Rechner nicht aus dem Internet erreichbar. Sicherer ist `MLL_HOST=127.0.0.1` und der Zugriff über einen SSH-Tunnel: `ssh -L 8080:localhost:8080 pi@raspberrypi.local`, danach im Browser `http://localhost:8080`.

9. Fehlersuche

9.1 Alle Systeme

Beobachtung	Ursache und Abhilfe
„Port 8080 ist belegt“	Läuft die Oberfläche schon? Dann öffnet das Symbol nur den Browser. Sonst anderer Port (`--port 8081` beim Start, auf dem Pi in der env-Datei)
Browser öffnet sich nicht	Adresse von Hand eingeben: `http://localhost:8080`. Der Start dauert auf langsamen Rechnern bis zu einer Minute, bevor der Browser sich öffnet: nicht noch einmal auf das Symbol klicken. Der Starter erkennt, dass die erste Instanz noch startet (Sperre im Datenordner), wartet bis zu 4 Minuten auf sie und öffnet dann den Browser
Nach dem Aktualisieren zeigt das Programm noch die alte Version (Titel der Oberfläche), auch nach einer Neuinstallation	Die alte Instanz läuft noch auf Port 8080, und der Starter öffnet nur den Browser auf ihr. Ab Version 1.25 sagt der Starter das (Terminal, Meldung auf dem Bildschirm und Protokoll: „ACHTUNG: Auf Port 8080 läuft noch eine Instanz ...“). Beenden: Knopf BEENDEN, sonst Linux/macOS `ps aux | grep webmobaledlib` und `kill <Nummer>` (oder `pkill -f webmobaledlib.py`), Windows `netstat -ano | findstr :8080` und `taskkill /PID <Nummer> /F`. Danach neu starten. Welche Version gerade läuft, zeigt `http://localhost:8080/mll-version`
Meldung „Connection lost“, die Seite baut sich immer wieder neu auf	Meist ein **langsamer Rechner**: Der Aufbau der Seite dauert Sekunden, und der Browser braucht danach noch einmal 10 bis 30 s, bis er sich mit dem Server verbindet. Der Server wartet nur die Zeit `MLL_RECONNECT_TIMEOUT` (Standard 60 s) auf den Browser; ist sie zu kurz, wird die Verbindung verworfen und die Seite

	neu geladen. Abhilfe: einen größeren Wert setzen (zum Beispiel `MLL_RECONNECT_TIMEOUT=180 install/start_linux.sh`), einen leichteren Browser benutzen, andere Programme schließen. Die Seiten Einstellungen und Farb Test werden erst beim ersten Öffnen aufgebaut (`MLL_LAZY_TABS=0` baut alles sofort auf), das verkürzt den Start auf langsamen Rechnern deutlich. Zur Diagnose im Protokoll (2.7) die Zeilen `Seitenaufbau: …` (wo die Zeit vergeht), `Browser verbunden` und `Browser getrennt` ansehen. Läuft der Server nicht mehr, im Terminal starten (Linux/Mac `install/start_linux.sh`, Windows `install\start_windows.bat`); `webmobaledlib.log` und `webmobaledlib-crash.log` (Datenordner, 2.6) schicken. Versuch auf Linux und Mac: ohne echtes Tk starten mit `MLL_TK_ROOT=0 install/start_linux.sh`
„Die Arduino Entwicklungsumgebung ist nicht … installiert“	IDE **1.8.19** installieren (2.2), ggf. den Pfad in „ARDUINO Einstellungen“ eintragen
Kompilieren: „MobaLedLib.h: No such file“ oder Bibliothek/Board fehlt	Bibliotheken installieren (2.3), in der Tabelle „Libraries“ prüfen
„Fehler beim Lesen der Datei …/preferences.txt“ (zum Beispiel bei „Check actual versions“)	Die Einstellungsdatei der IDE fehlt auf einem frischen System (Linux, macOS, Pi). Das Programm legt sie beim Start an. Von Hand (Linux/Pi): `mkdir -p ~/.arduino15 ~/Arduino/libraries; printf 'sketchbook.path=%s/Arduino\ntarget_package=arduino\ntarget_platform=avr\nboard=uno\n' "$HOME" > ~/.arduino15/preferences.txt`
Board installieren: `NullPointerException at … BaseNoGui.onBoardOrPortChange`	Die IDE hat ein nicht installiertes Board ausgewählt. Das Programm und die Skripte setzen die Auswahl auf `arduino:avr:uno` zurück. Von Hand (Linux/Pi): `sed -i -E '/^(target_package|target_platform|board|custom_[A-Za-z0-9_.]+)=/d' ~/.arduino15/preferences.txt; printf 'target_package=arduino\ntarget_platform=avr\nboard=uno\n' >> ~/.arduino15/preferences.txt`, dann wiederholen
Board `esp32:esp32` installieren: `Tool esptool_py is not available for your operating system`	Für diese Version des ESP32-Pakets fehlen Werkzeuge für die Prozessorart (64-Bit-ARM), siehe Kapitel 10
Pakete lassen sich nicht installieren	Internetverbindung, Firmenproxy; im Fenster steht die Fehlermeldung von `pip`
Beim Installieren der Python-Pakete: `Failed to build <Paket>` oder `Failed to build installable wheels`	Für dieses System gibt es keine fertige Datei (Wheel) des Pakets, und pip versucht, es zu übersetzen, was meist scheitert. Häufigste Ursachen: 32-Bit-System, sehr altes oder sehr neues Python, kein Internet. Das Linux-Skript zeigt eine Diagnose (Prozessor, Python, pip, glibc): bitte mitschicken. Pillow wird nicht mehr gebraucht (`requirements-web.txt` ab Version 1.01); bei einer älteren Fassung die Zeile `Pillow` aus `install/requirements-web.txt` löschen und das Skript noch einmal ausführen
„ARDUINO IDE nicht gefunden“ (Protokoll, Meldung beim Start, kein Kompilieren möglich)	Siehe 2.2. Die IDE 1.8.19 nach `~/arduino-1.8.19` legen, oder in „ARDUINO Einstellungen“ **IDE suchen** benutzen bzw. den vollständigen Pfad der Startdatei `arduino` eintragen. Es muss die IDE **1.8.x** sein, nicht 2.x. Die Orte, an denen gesucht wurde, stehen im Protokoll
`ModuleNotFoundError: No module named 'xyz'`	`venv_web/bin/pip install xyz` (Windows `venv_web\Scripts\pip`, Pi `venv/bin/pip`)

Konfiguration wird nicht gespeichert	Ist der Datenordner (2.6) beschreibbar? Ist er es nicht, nimmt das Programm einen Ersatzordner im temporären Verzeichnis und warnt im Protokoll. Anderen Ort einstellen: `MLL_DATA_DIR`

9.2 Windows

Beobachtung	Ursache und Abhilfe
„Python wurde nicht gefunden“ oder „Python wurde installiert, ist aber in diesem Fenster noch nicht bekannt“	**PC neu starten** (oder ab- und wieder anmelden) und das Skript noch einmal starten. Python von Hand: python.org, dabei „Add python.exe to PATH“ ankreuzen
„Der Computer wurde durch Windows geschützt“	*Weitere Informationen → Trotzdem ausführen*
Fenster geht sofort wieder zu	`install\start_windows.bat` in einer Eingabeaufforderung starten und die Meldung lesen; außerdem `webmobaledlib.log` im Datenordner (2.6)
Verknüpfung fehlt auf dem Desktop	Pfad des Projektordners ohne `'` und `&`? Sonst `install\start_windows.bat` per Rechtsklick → *Senden an → Desktop*
IDE-Installation (`/ide`) bricht ab	Download von arduino.cc von Hand (2.2), danach `install_windows.bat /libs`

9.3 macOS

Beobachtung	Ursache und Abhilfe
„… kann nicht geöffnet werden, da es von einem nicht verifizierten Entwickler stammt“	Rechtsklick auf `pyMobaLedLib` → *Öffnen* → *Öffnen* (nur beim ersten Mal)
`install_mac.command` lässt sich nicht öffnen	`chmod +x install/install_mac.command`, oder im Terminal `bash install/install_mac.command`
IDE startet nicht (Apple-Chip)	Rosetta installieren: `softwareupdate --install-rosetta`
Programm startet nichts	`install/start_mac.command` im Terminal ausführen und die Meldung lesen; `webmobaledlib-console.log` im Datenordner (2.6)

9.4 Linux-Desktop

Beobachtung	Ursache und Abhilfe
„Python 3.10 oder neuer wurde nicht gefunden“	System zu alt (Mint 20, Debian 11): neueres Python über pyenv oder deadsnakes installieren, das Skript noch einmal starten
`venv fehlgeschlagen` / `ensurepip is not available`	`sudo apt install python3-venv python3-pip`
Symbol startet nichts	`install/start_linux.sh` im Terminal ausführen und die Meldung lesen; Symbol ausführbar? (`ls -l ~/Desktop/pyMobaLedLib.desktop`)
Kein serieller Anschluss / „permission denied“ auf `/dev/ttyACM0`	Gruppe `dialout`: `groups` muss `dialout` enthalten, sonst ab- und wieder anmelden (oder `sudo usermod -aG dialout $USER`)

9.5 Raspberry Pi

Beobachtung	Ursache und Abhilfe
Symbol: „Der Dienst konnte nicht gestartet werden“	sudo-Regel fehlt: `sudo bash install_pi.sh --no-check` noch einmal, oder `sudo systemctl start mobaledlib-web`
Symbol: keine Reaktion	im Terminal `~/mobaledlib/python/pi/start_mobaledlib.sh` ausführen und die Meldung lesen; Symbol ist ausführbar? (`ls -l ~/Desktop`)
Seite lädt nicht	`systemctl status mobaledlib-web`, Adresse und Port prüfen (Port `:8080` an die Adresse anhängen), beim ersten Start 1 Minute warten
`raspberrypi.local` wird nicht gefunden	mDNS geht nicht überall: Gastnetz oder WLAN mit Geräteisolation, Firmennetz, Android. Die IP-Adresse nehmen (Router, `find_mobaledlib.py`, 6.7); auf dem Pi `systemctl status avahi-daemon`
Anmeldefenster, Passwort wird nicht akzeptiert	Benutzer `mobaledlib`; Passwort: das Standardpasswort `mobaledlib` oder das im Programm gesetzte. Vergessen: Datei `web_password.json` im Datenordner löschen (Kapitel 8)
Kein Port in der Auswahl / „permission denied“ auf `/dev/ttyACM0`	Gruppe `dialout` (der Dienst hat sie), bei Tests von Hand einmal ab- und anmelden
Kompilieren: „no X11 DISPLAY“ oder Java-Fehler	Läuft der Dienst über `xvfb-run`? (`/etc/systemd/system/mobaledlib-web.service`, `sudo apt install xvfb`)
Der Dienst startet nicht	`journalctl -u mobaledlib-web -e`; `pi_check.py` zeigt fehlende Pakete
Anmeldung am Pi per Netz geht nicht	`MLL_HOST=0.0.0.0` und Passwort in `/etc/mobaledlib-web.env` (Kapitel 8)

10. Was nicht oder noch nicht geht

- **ESP32 auf 64-Bit-ARM (Raspberry Pi, vermutlich auch Macs mit Apple-Chip):** Das Programm verlangt das ESP32-Boardpaket in der Version 1.0.4. Für diese Version bietet Espressif keine Werkzeuge für diese Prozessorart an, die Installation über die Boardverwaltung der IDE 1.8.19 scheitert mit „Tool esptool_py is not available for your operating system“. ESP32-Programme werden deshalb auf einem Windows- oder Intel-Rechner erzeugt und hochgeladen. Pico und Arduino laufen auch auf dem Pi. `python3 pi/esp32_index_check.py <Version>` zeigt, für welche Versionen alle Werkzeuge für den Rechner vorhanden sind. Eine neuere Version müsste das Projekt festlegen und testen.
- **Nur in Tk vorhandene Formulare** (ServoAnim, LEDAnim, LEDColorAnim, Multiple_LEDColorAnim, Mustereditor) zeigen im Web nichts an.
- **Weitere Reiter** (ARDUINO Monitor, Sound Test, DCC Keyboard, Servo Test) gibt es im Web noch nicht.
- **Stand der Prüfung:** Ausprobiert sind
  - **Raspberry Pi** (Raspberry Pi OS Trixie, Python 3.13): Neuinstallation mit `install_pi.sh`, Dienst, Zugriff aus dem Netz, Desktop-Symbol, IDE, Bibliotheken, Boards (ohne ESP32) und das Herunterfahren.
  - **Linux Mint** (32-Bit-Debian 12, Python 3.11, sehr langsamer Laptop) und **Ubuntu 25.10** (Python 3.13): Installation mit `install_linux.sh`, Betrieb, Update über das Menü und das kompakte Layout.
  - **Windows 10** (`install_windows.bat /ide` nach dem Neustart des PCs, Python 3.12) und **Windows 11** (Python 3.14): Installation und Betrieb.

    - **Nicht ausprobiert: macOS** (auf macOS 10.11.6 scheitert die Installation der Python-Pakete, `orjson`; neuere Versionen sind nicht getestet) und der Download und die stille Installation der IDE unter Windows und macOS. Die Skripte für macOS liefen nur unter Linux mit einer lokalen ZIP-Datei als Ersatz.
    - **32-Bit-Systeme** funktionieren nur, wenn für alle Python-Pakete fertige Dateien vorliegen (Pillow wird nicht mehr gebraucht). Auf dem Mint-Laptop war das der Fall.
- **Update über das Programm** gibt es ab Version 1.27. Beim ersten Update einer Installation werden keine veralteten Dateien gelöscht (7.1).
- **Kompaktes Layout:** Die Zeilenhöhe der Tabelle bleibt unverändert (22 Pixel), Dialoge sind nur über Schrift und Abstände angepasst.

Bei einer Fehlermeldung bitte den Text aus dem Installationsfenster und die Dateien `webmobaledlib.log` und `webmobaledlib-crash.log` aus dem Datenordner (2.6) schicken.